



北京航空航天大学
BEIHANG UNIVERSITY



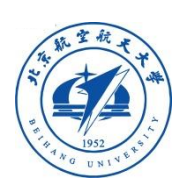
微机原理与接口技术

第六章 (1)

林新

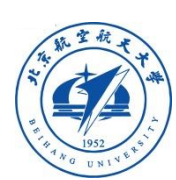
Lx@buaa.edu.cn

北京航空航天大学 自动化学院



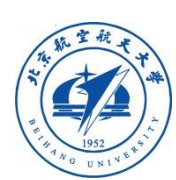
主要内容

- 输入输出接口 (IO接口)
 - 数据输入/输出的概念：接口的概念及其基本功能；端口的概念；数据端口、状态端口、命令端口；I/O信号的类型
 - 数据的输入/输出方法：程序控制（无条件传送、查询）方式，程序中断方式，直接存贮器存取方式（DMA）
- 并行接口芯片8255A
 - 掌握8255A的工作方式0
 - 掌握利用8255A进行键盘识别、显示的方法



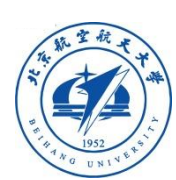
基本要求

- 掌握接口的概念
- 掌握接口所包含的常用信号线
- 了解接口的编址方法
- 掌握输入/输出指令
- 熟悉基本的数据输入/输出方法
- 8255A引脚、方式控制字、工作方式0、工作方式设置



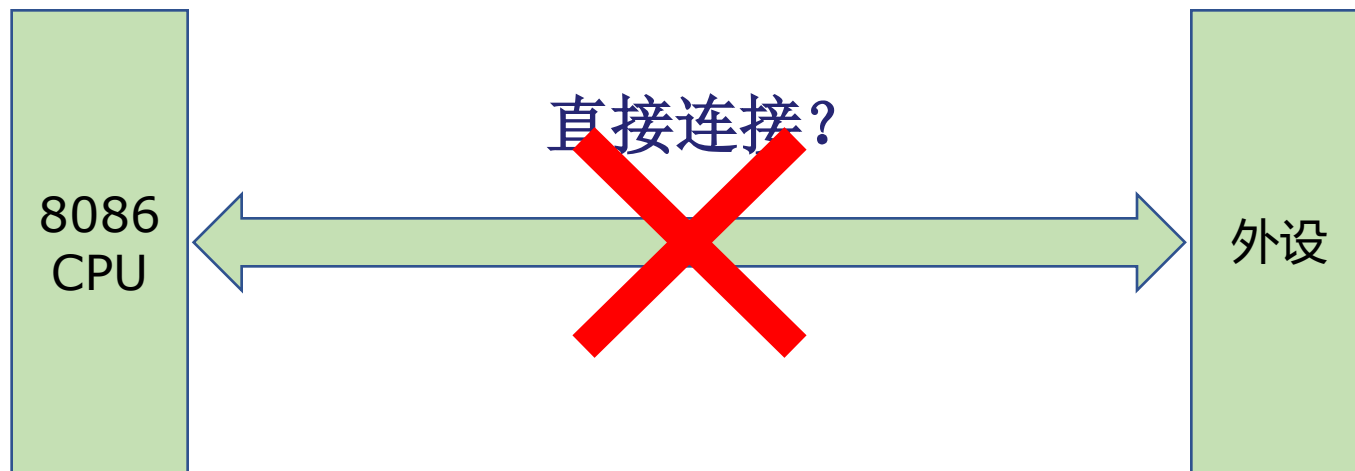
6.1 I/O接口

- 信息的输入或输出设备，称为外部设备（Peripherals），简称外设或IO设备
 - 常用的外围设备有：键盘、显示器、磁带机、硬盘驱动器、软盘驱动器、模数转换器、数模转换器、调制解调器以及一些专用设备等
- 计算机与外设之间交换数据、状态和控制命令的过程称为通信。

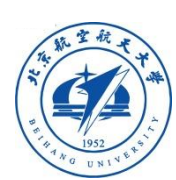


6.1 I/O接口

计算机与外设如何连接？



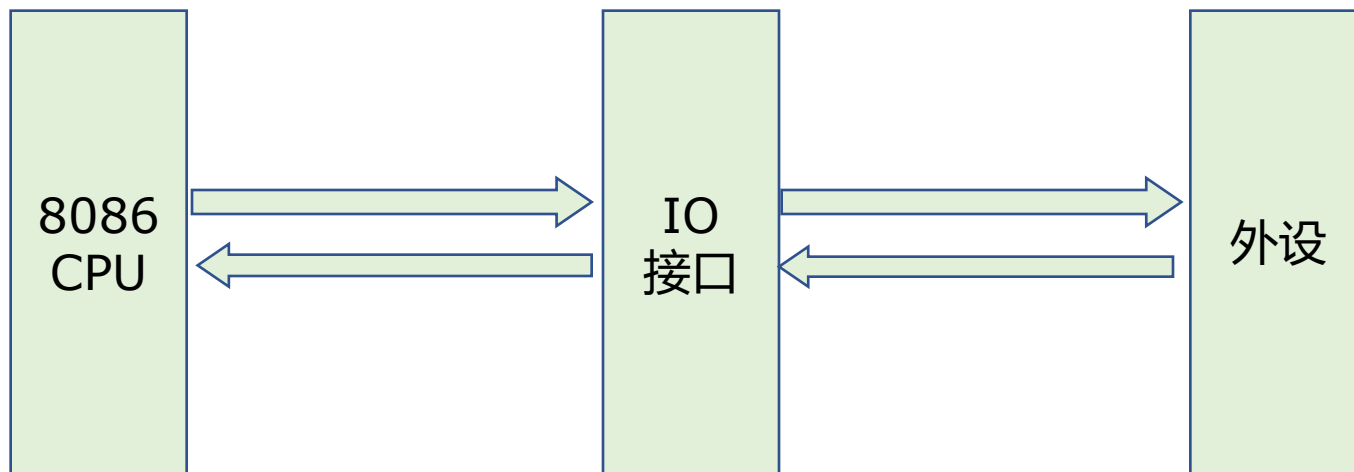
- 计算机与外设交换信息主要问题：
 - 速度不匹配
 - 信号电平不匹配
 - 信号格式不匹配
 - 时序不匹配

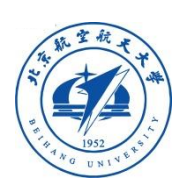


6.1 I/O接口

计算机与外设如何连接？

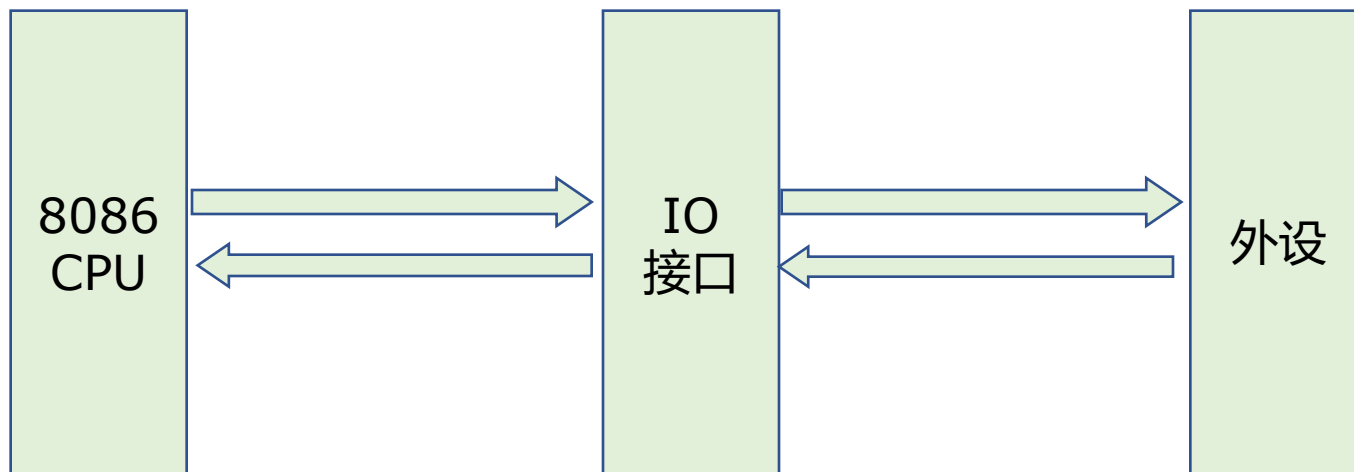
加入IO接口电路 → 协调CPU和外设

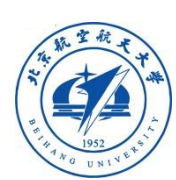




6.1 I/O接口

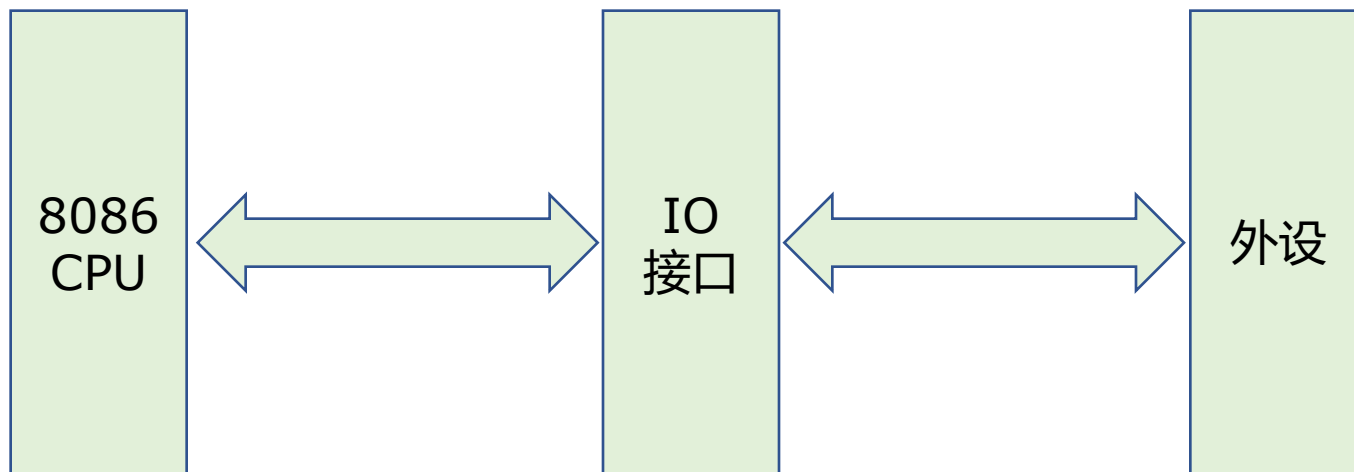
加入IO接口电路 → 协调CPU和外设

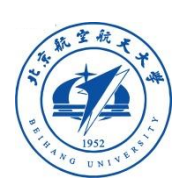




6.1 I/O接口

加入IO接口电路 → 协调CPU和外设



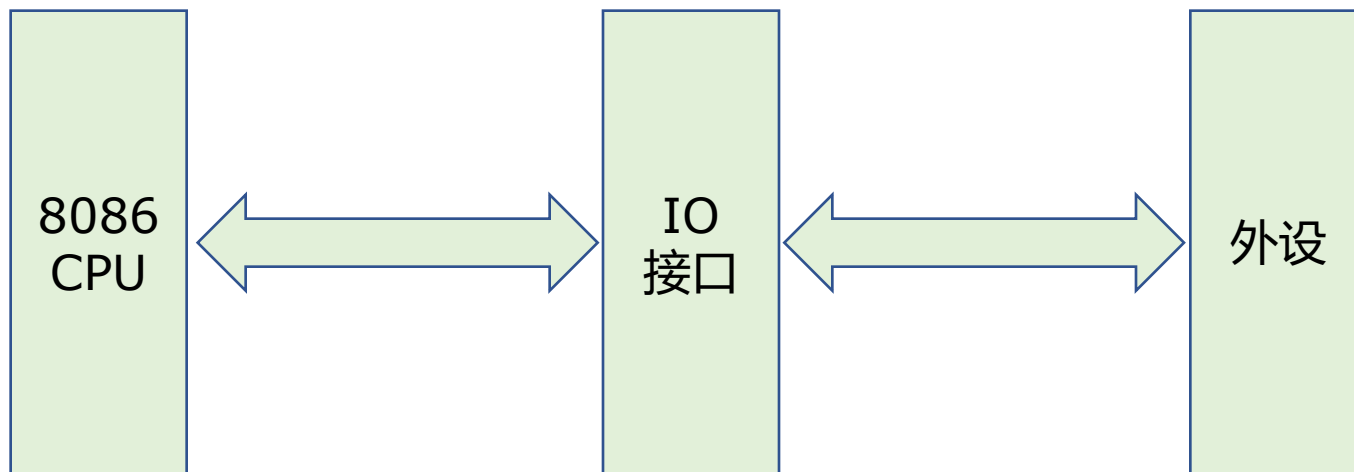


6.1 I/O接口

接口电路的作用是把计算机输出的信息变成外设能够识别的信息，把外设输入的信息转化成计算机所能接受的信息，它使得外部设备和计算机之间能够很好的协调工作。

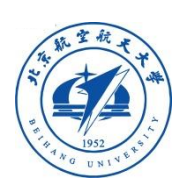
加入IO接口电路

协调CPU和外设



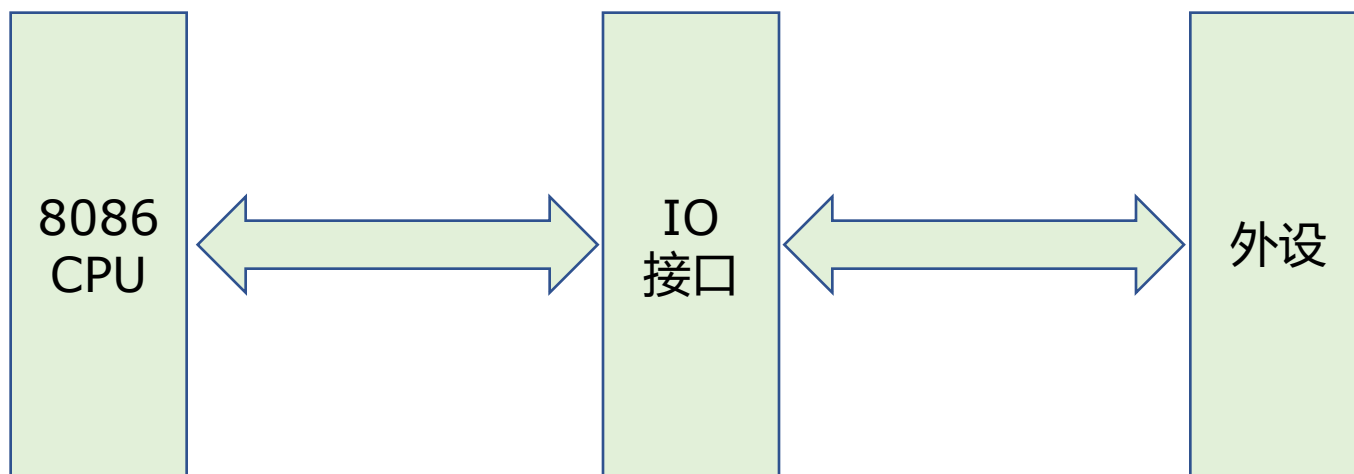
- 设置数据缓冲，匹配速度差异
- 电平转换

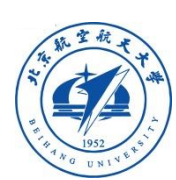
- 格式转换
- 时序同步
- 地址译码



6.1 I/O接口

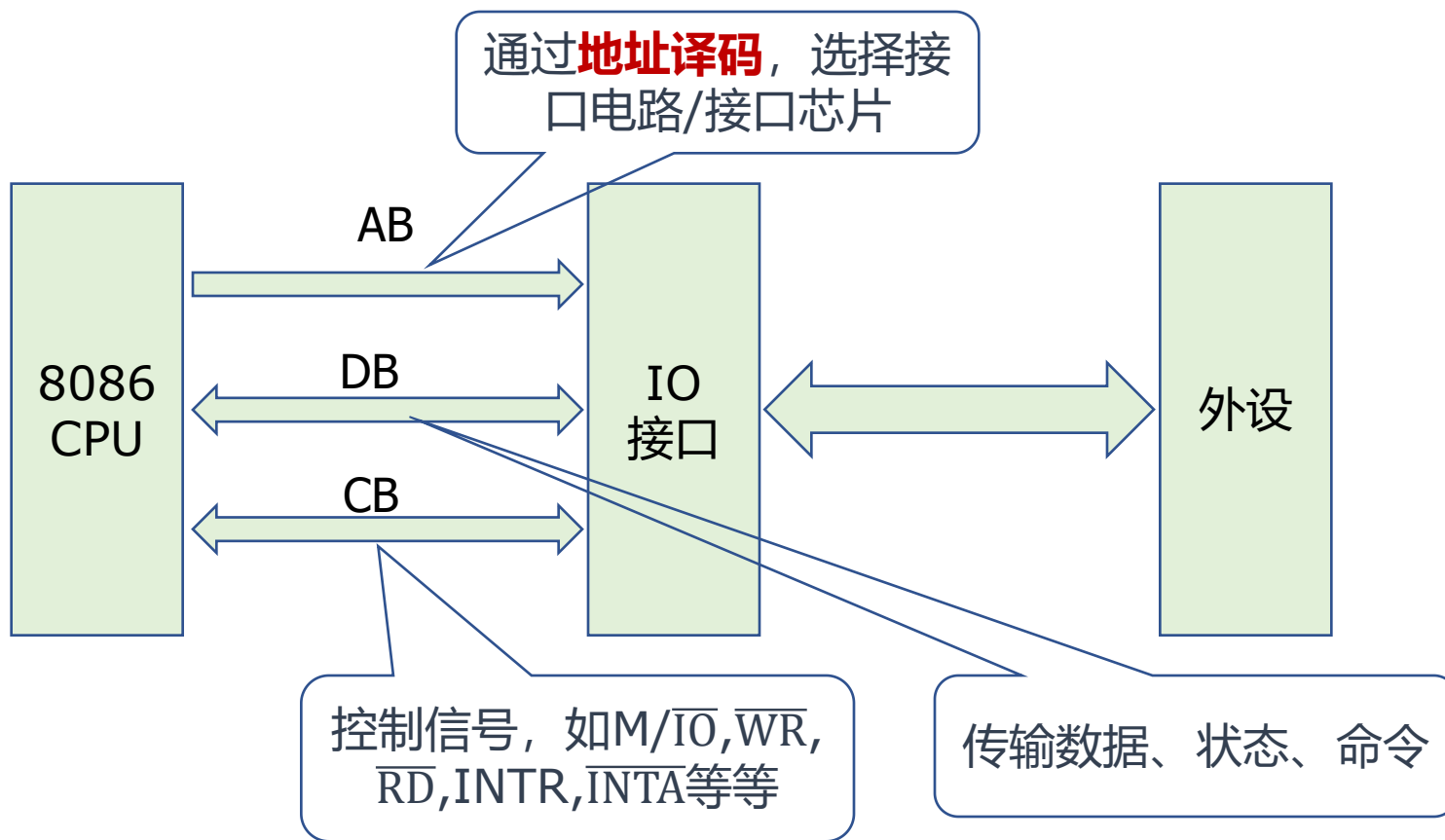
因此：I/O接口是一组电路，用于连接微机系统和外部设备，使得CPU能够与外设交换数据





6.1 I/O接口

I/O接口与CPU都是通过数据总线、地址总线、以及控制总线连接



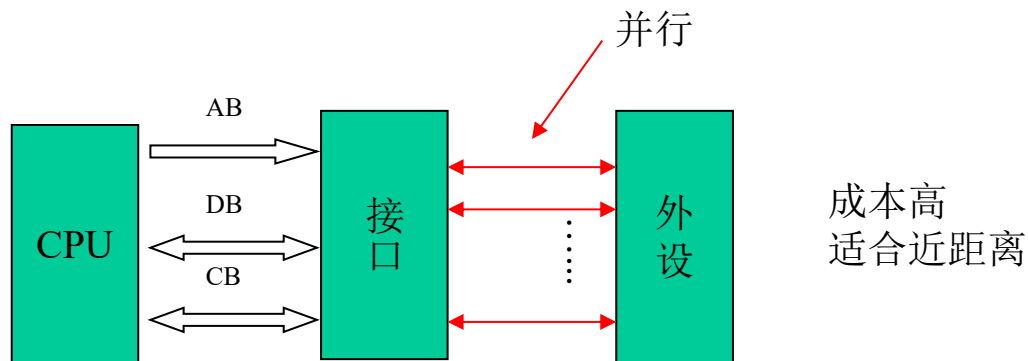
I/O接口与CPU连接



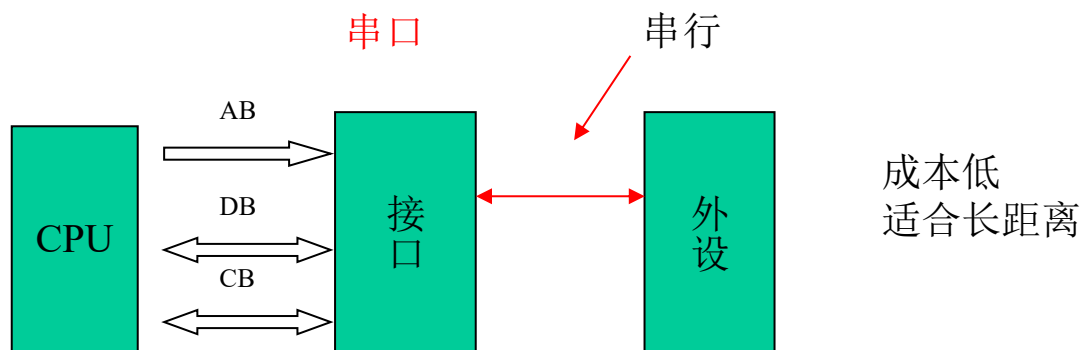
6.1 I/O接口

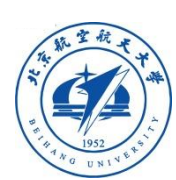
I/O接口与外设连接的方式

■ 并行

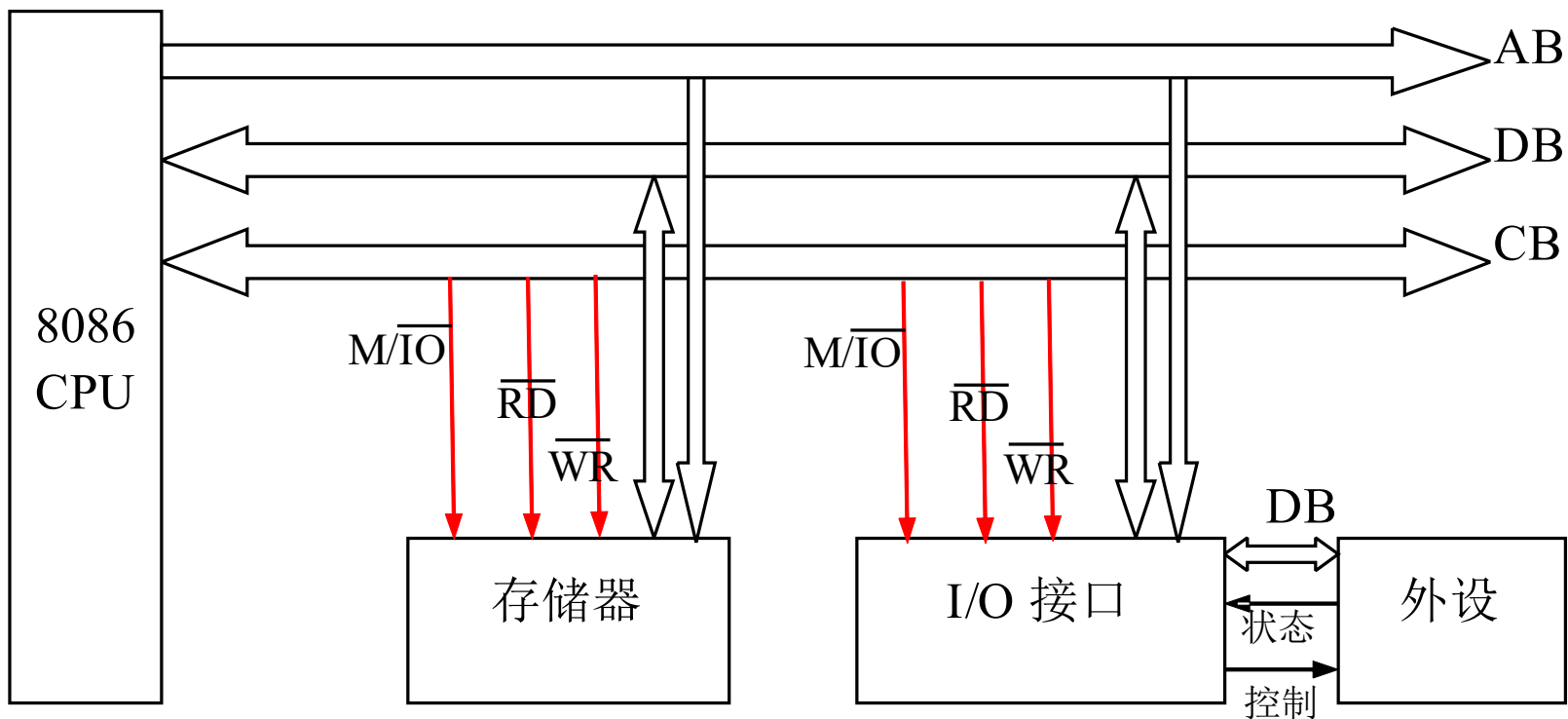


■ 串行

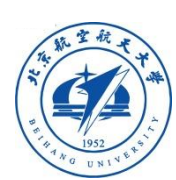




6.1 I/O接口

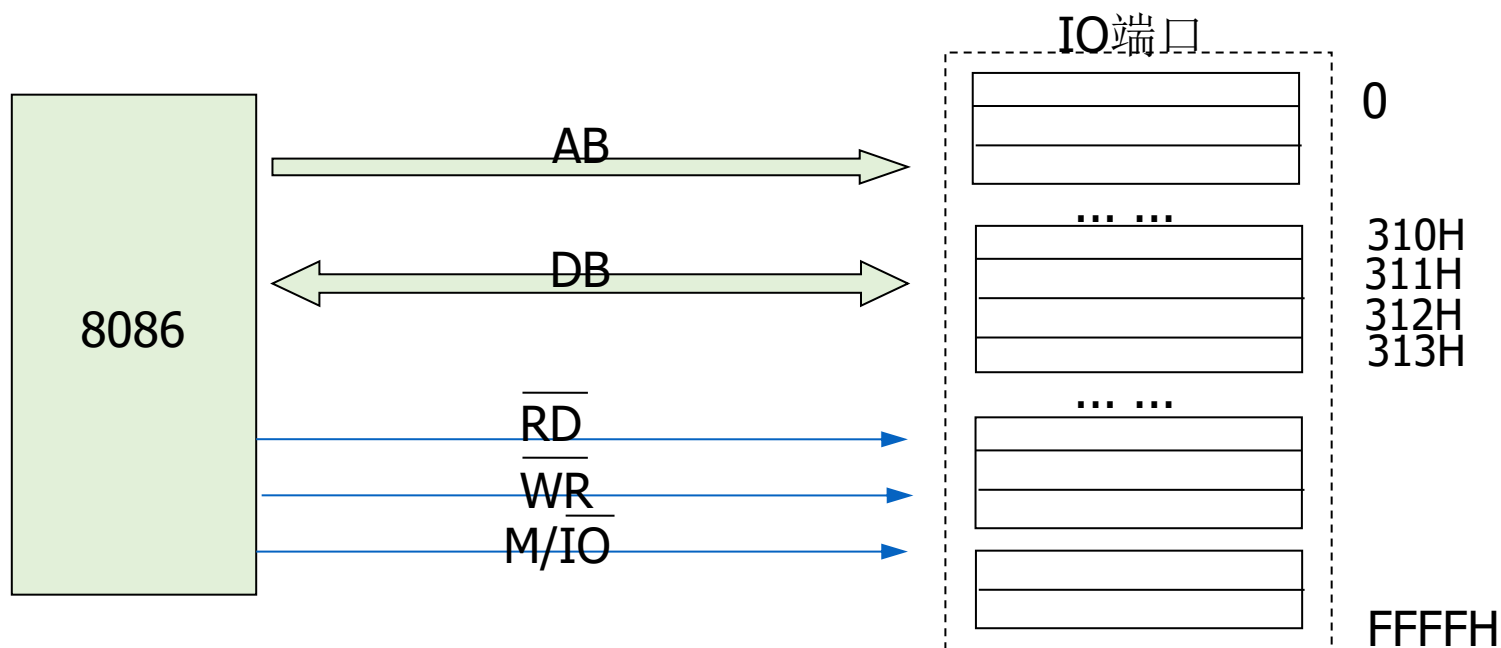


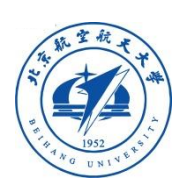
I/O接口与CPU都是通过数据总线、地址总线、以及控制总线连接



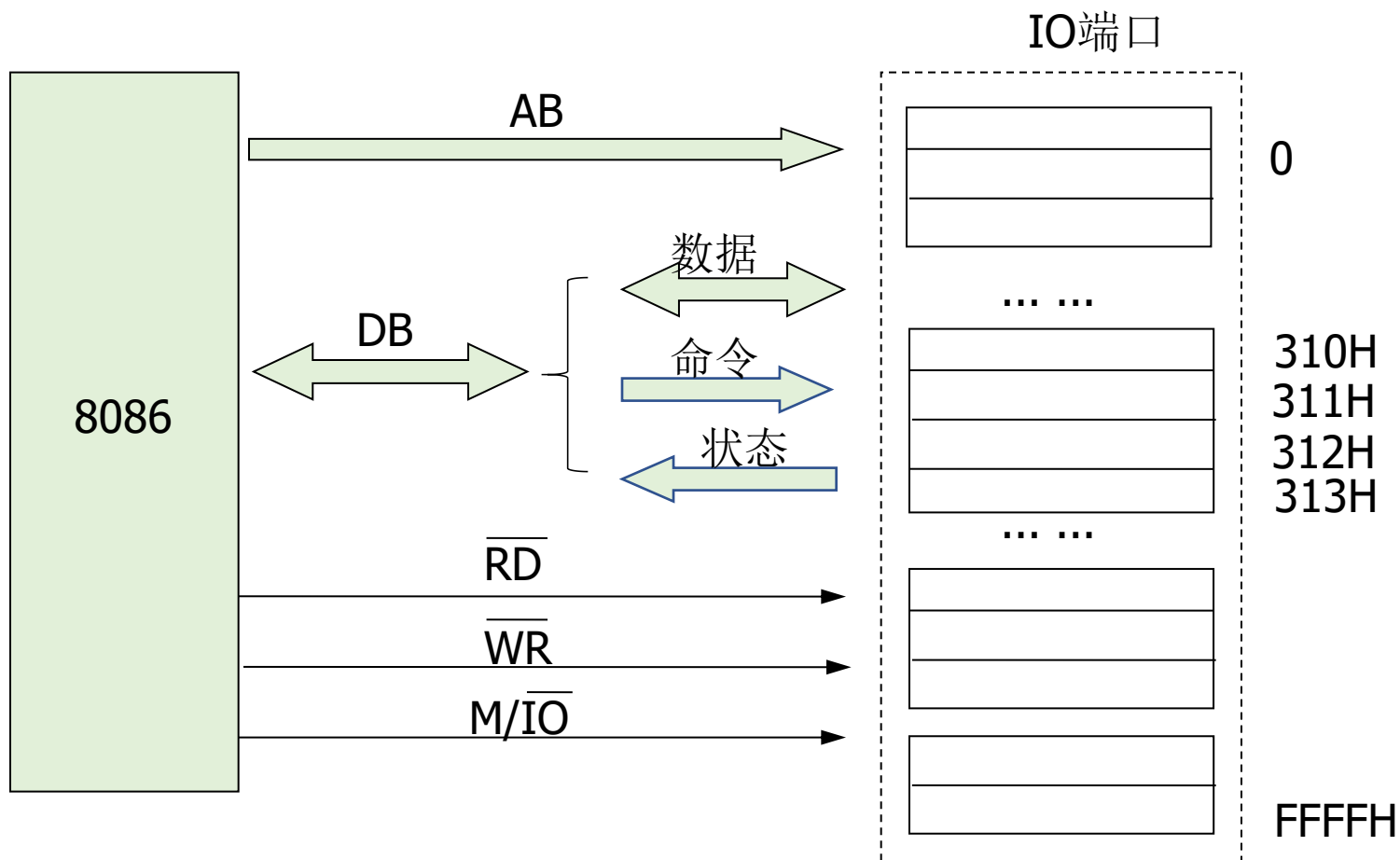
6.1.2 I/O端口

- **端口**：IO接口电路中，用于和CPU交换数据的地方，通常是寄存器或者数字电路。
- CPU对端口的读写操作类似于存储单元

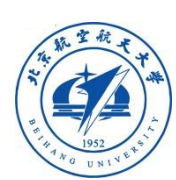




6.1.2 I/O端口-端口类型



☆端口 ☆ CPU输出数据 ☆ CPU读入数据



6.1.2 I/O端口-端口类型

CPU与IO接口电路之间交换的信息有三类：数据信息、控制信息和状态信息，因此端口类型也相应划分为三种：

1. **数据端口**：用于和CPU交换数据信息
 - 数字量，模拟量，开关量
2. **命令端口/控制端口**：用于接收来自CPU的命令或者控制信息。
3. **状态端口**：用于提供IO电路或者外设的工作状态。
常用状态位：
 - 就绪 (READY)
 - 忙碌 (BUSY)
 - 错误 (Error)



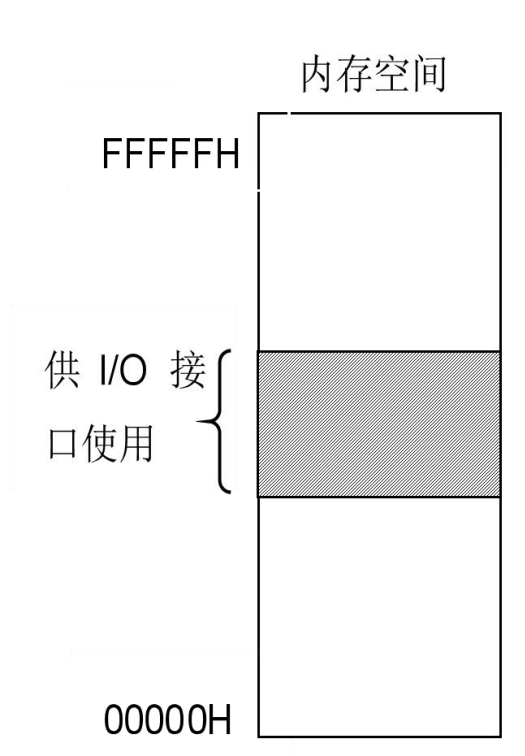
6.1.3 端口寻址方式

计算机系统中通常存在着多个I/O端口。为此，需要给每一个端口进行**编号**（称为**编址**）。通常有以下两种编址方式

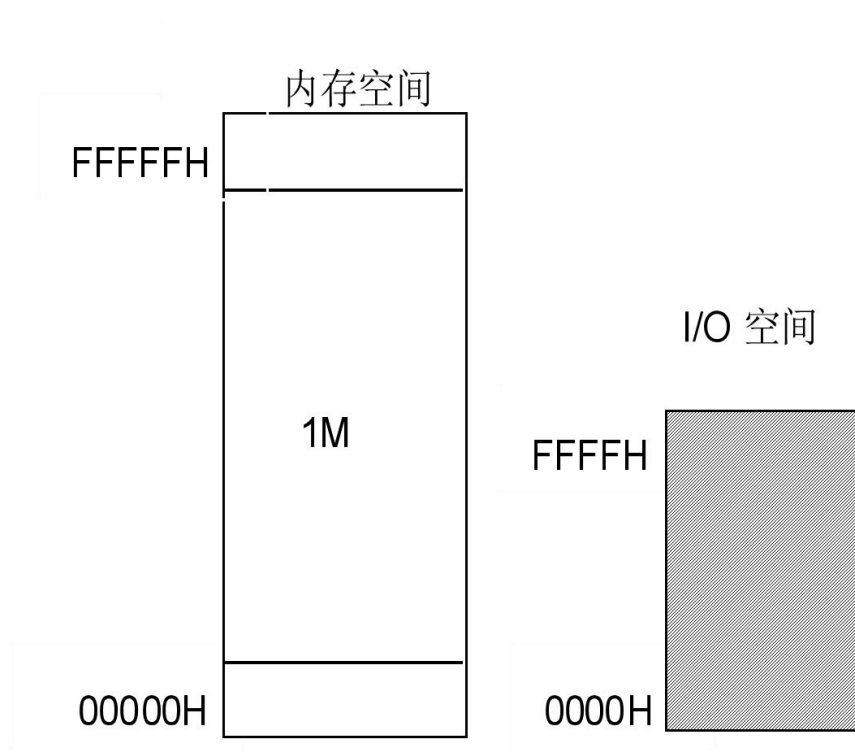
- **存储器影像寻址**：I/O 端口与存储器统一编址
- **I/O 端口单独编址**：专门I/O指令，独立的端口地址



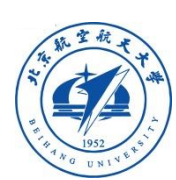
6.1.3 端口寻址方式



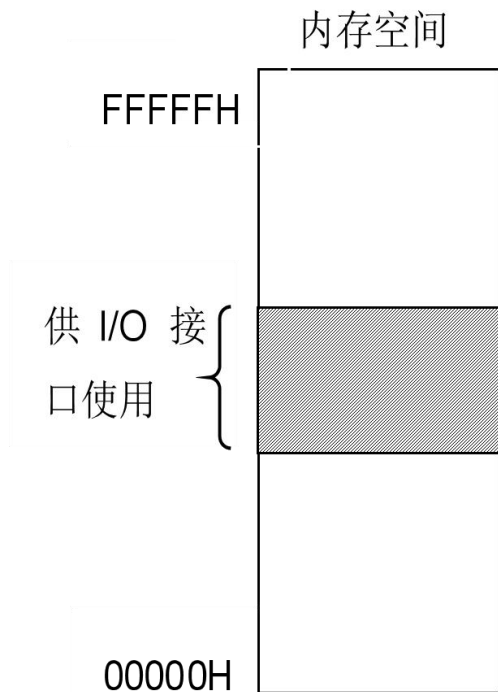
a) 存储器映像寻址（存储器映射方式）



b) I/O 单独编址（IO 映射方式）



6.1.3 端口寻址方式



a) 存储器映像寻址（存储器映射方式）

存储器影像寻址

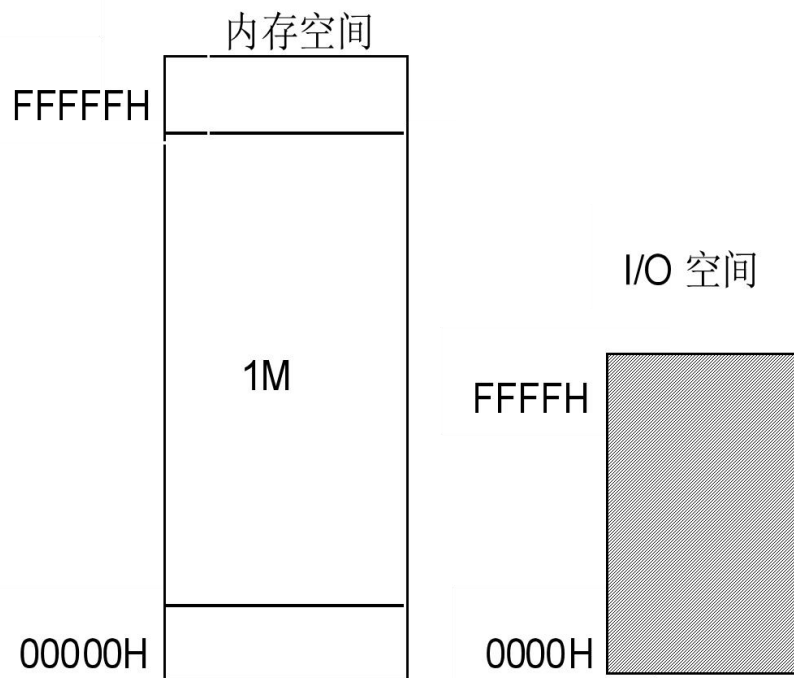
- 系统中的每个I/O端口都看作一个存储单元，并与存储单元一样统一编址。
- 访问存储器的指令都可以用来访问I/O端口，不用设置专门的I/O指令
- 典型的CPU有Motorola的MC6800, MC68000, ARM系列芯片等等



6.1.3 端口寻址方式

I/O 端口单独编址

- 系统中的I/O端口地址单独编址，构成一个**I/O空间**，独立于**存储空间**之外。
- 对I/O端口的操作也需要使用专门的指令来完成
- 8086采样此种方式

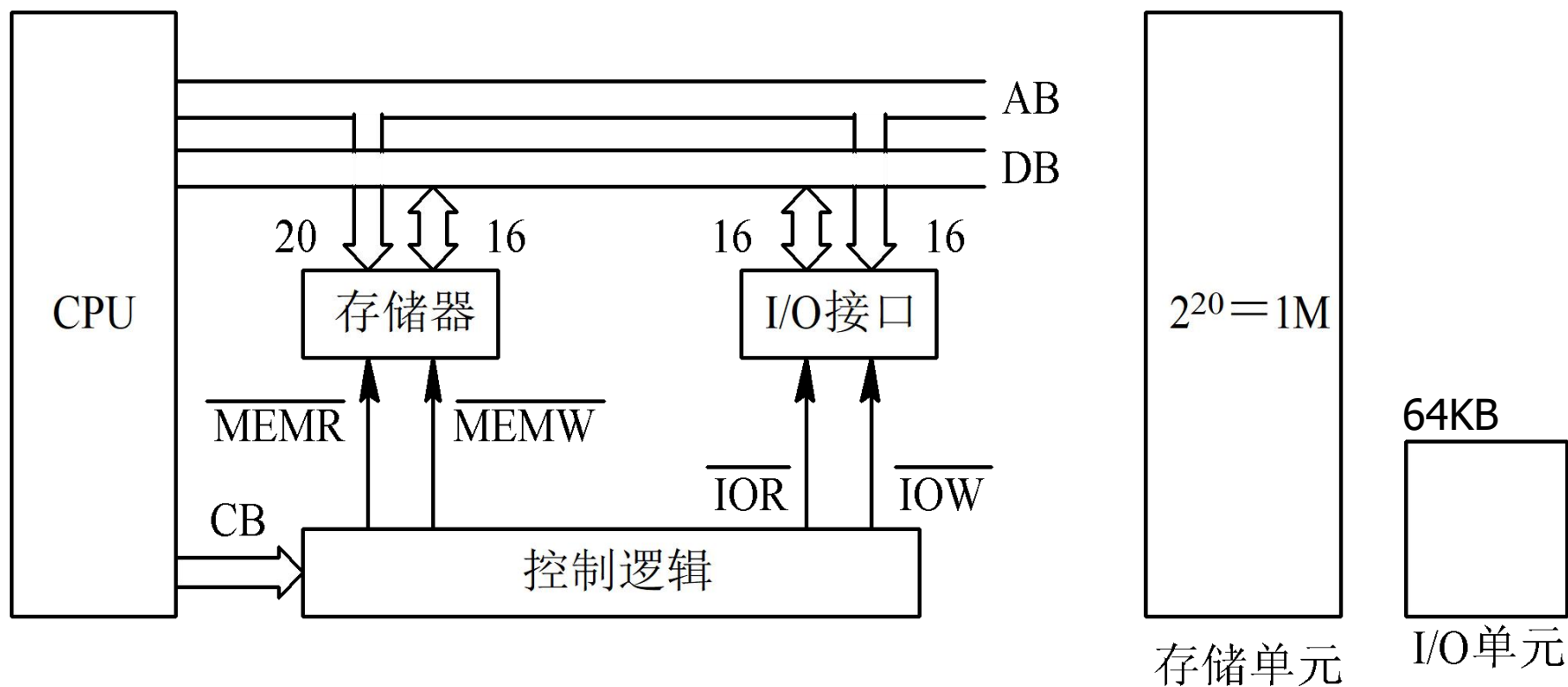


b) I/O 单独编址 (IO 映射方式)

8086存在两个地址空间:

- 1) **I/O空间**
- 2) **存储空间**

8086的I/O空间及地址范围



- 8086系统使用地址总线的**低16 (A15~A0) 位**来寻址I/O端口，最多可以访问 $2^{16} = 65536$ 个输入或输出端口
- CPU中的**M/ $\overline{IO}$** (或 **$\overline{IOR}$ / $\overline{IOW}$**)控制信号用来区分是I/O寻址还是存储器寻址



6.1.3 端口寻址方式

■ I/O指令寻址方式的优点

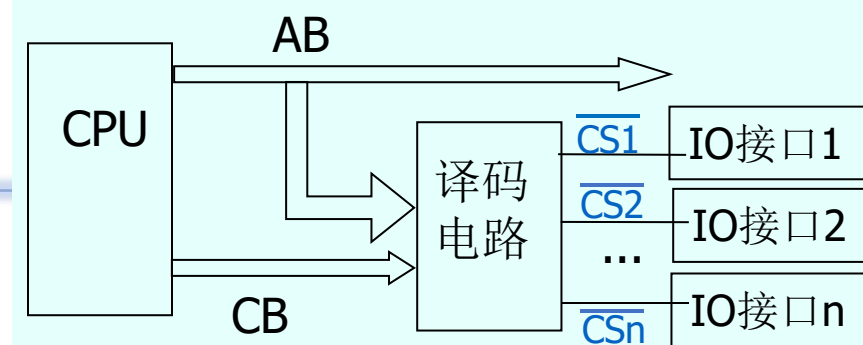
- I/O端口的地址码一般比同系统中存储单元的地址码短，译码电路较简单
- 存储器同I/O端口的操作指令不同，程序比较清晰
- 存储器和I/O端口的控制电路结构相互独立，可以分别设计

■ 缺点

- 需要专门的I/O指令，这些I/O指令一般没有存储器访问指令丰富，所以程序设计的灵活性较差



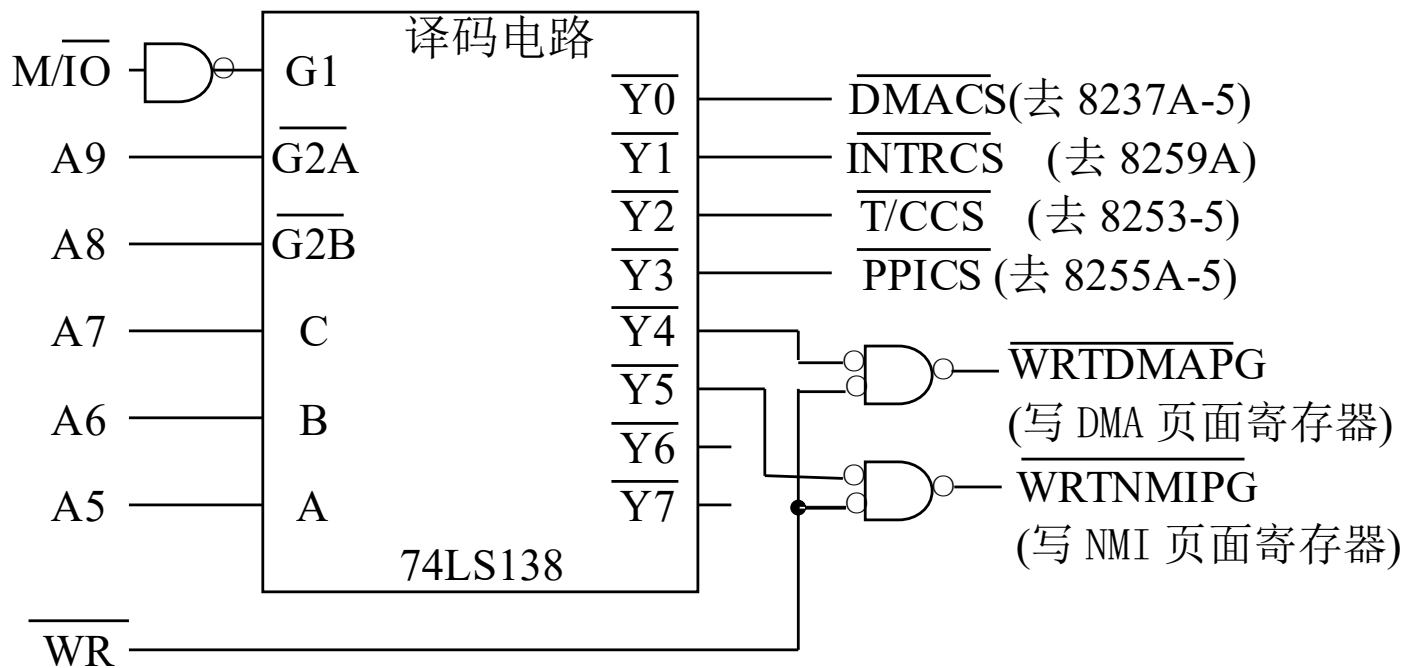
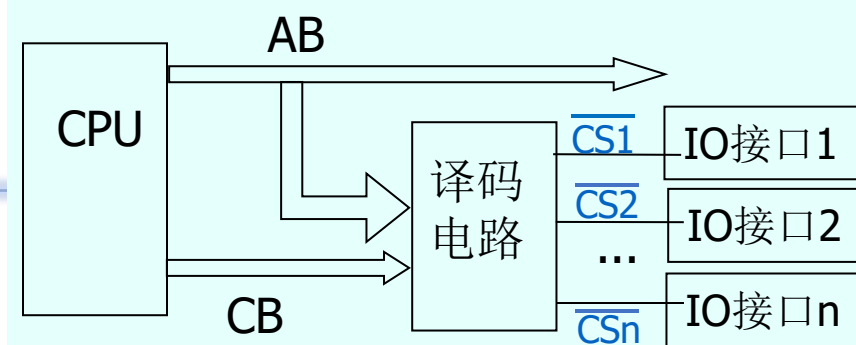
补充：译码电路



- 设计一个数字电路，使得仅当4条输入信号 $A3, A2, A1, A0$ 为0101B（即5H）时，输出信号 $\overline{CS}$ 为0，否则输出为1



补充： 地址译码电路芯片

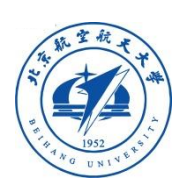


- M/I0, A9, A8 选择 74LS138
- A7~A5, 译码输出 Y0~Y7
- A4~A0, 选择 I/O 芯片内部端口



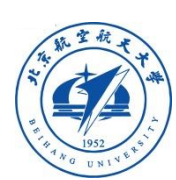
6.1.3 CPU与外设数据的交互

- CPU与外设数据传送的三种方式
 - 程序方式：无条件传送，查询方式
 - 中断方式
 - 存储器直接存取方式（DMA方式）



6.1.3 CPU与外设数据的交互-程序方式

- **程序方式**是指：（仅仅）由**程序直接控制**CPU与外设之间的数据传递。可以分为**无条件传送**和**查询方式**两种



6.1.3 CPU与外设数据的交互-程序方式

1) 无条件传送

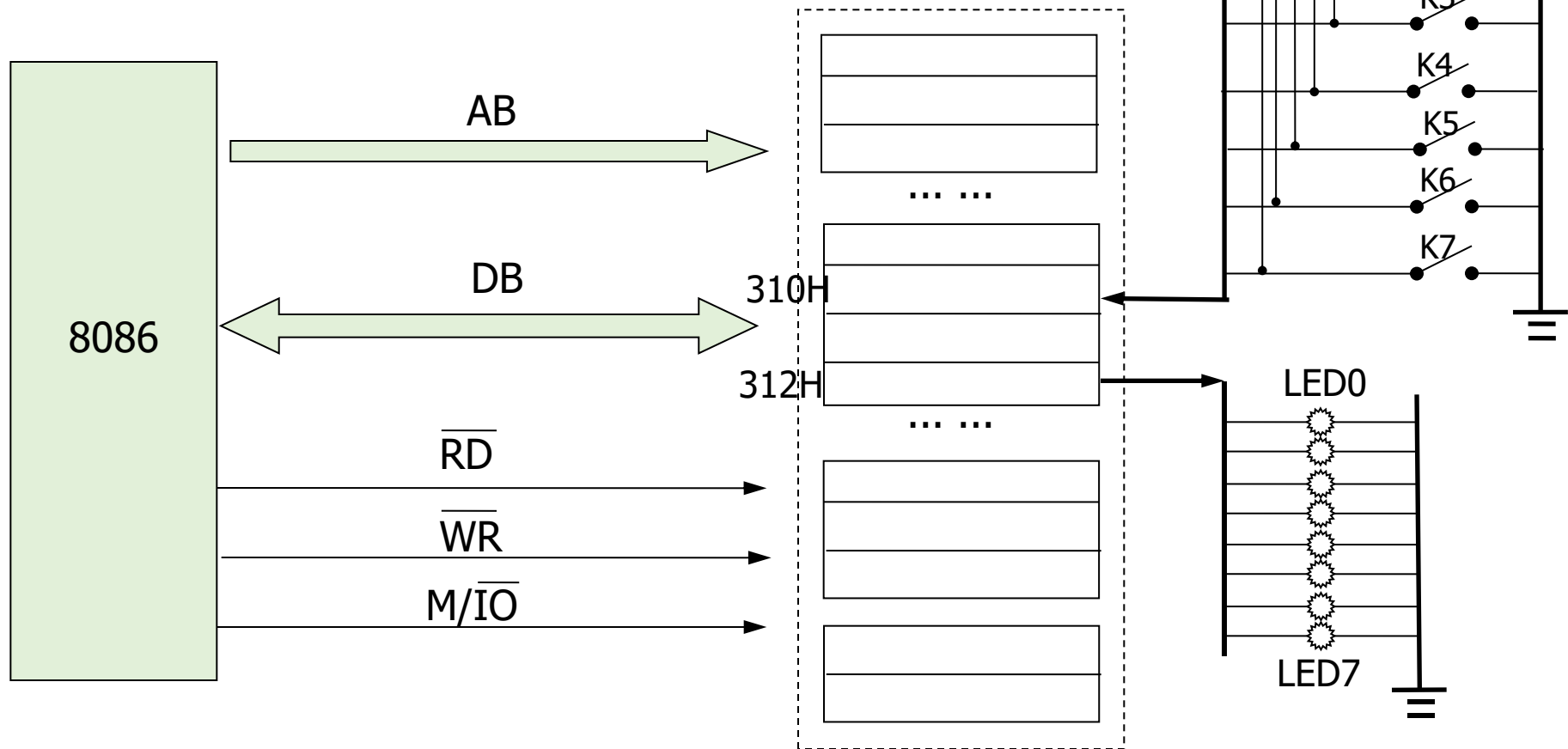
- 也称为同步传送方式。在该方式中，**外设**被认为**总是处于就绪**状态。当需要交换数据时，程序直接执行输入输出指令，开始接收和发送数据

第三章习题：TEST指令

设系统如图所示，开关状态（断开代表逻辑1，闭合代表逻辑0）被自动送往IO端口310H，K0对应310H端口的第0位，K7对应第7位，其它依此类推。
IO端口312H中的数据自动送往发光二极管LEDx，每一位都和一个LED灯相对应。如312H端口的第0位为1时LED0亮，为0时LED0灭，其它依此类推。

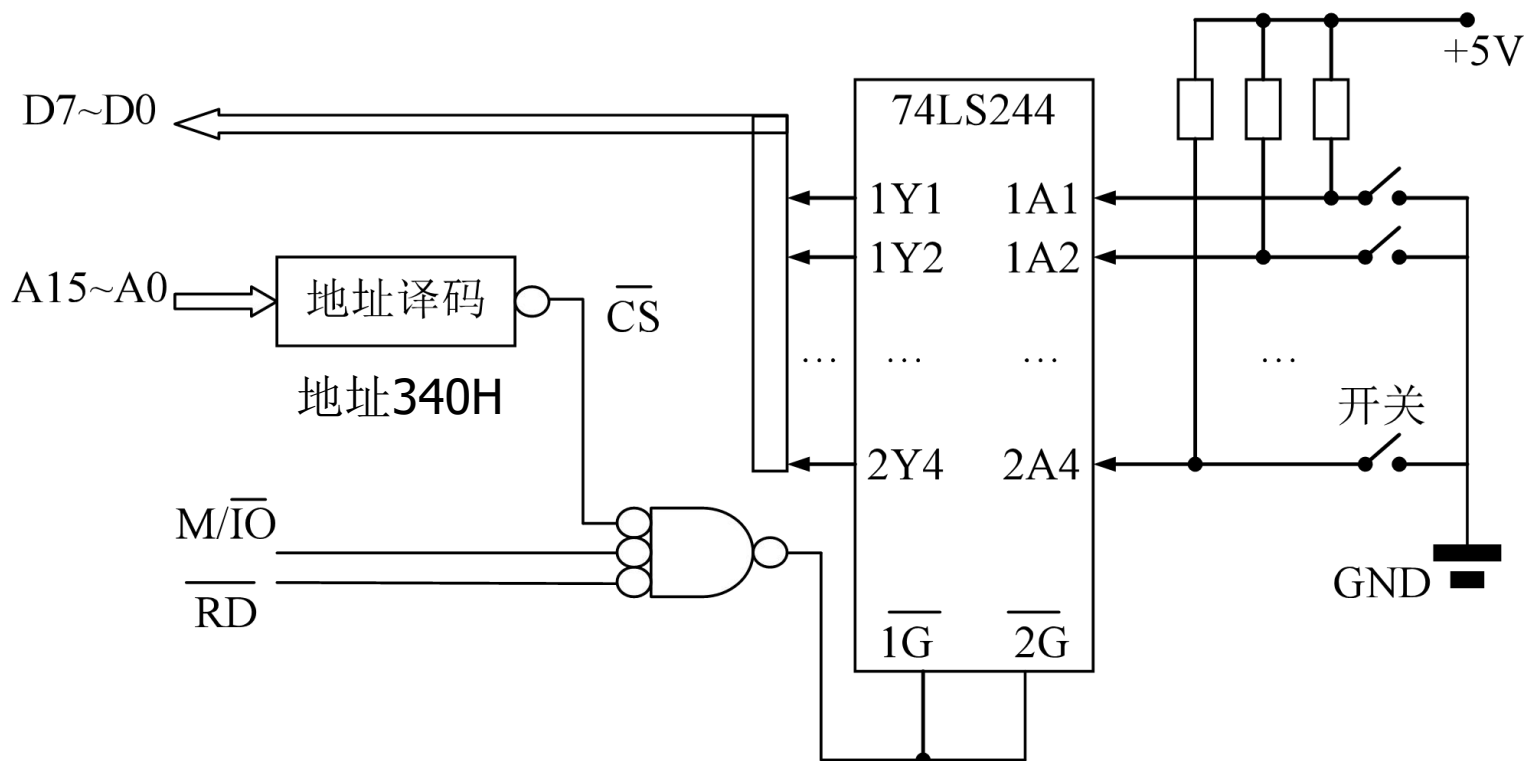
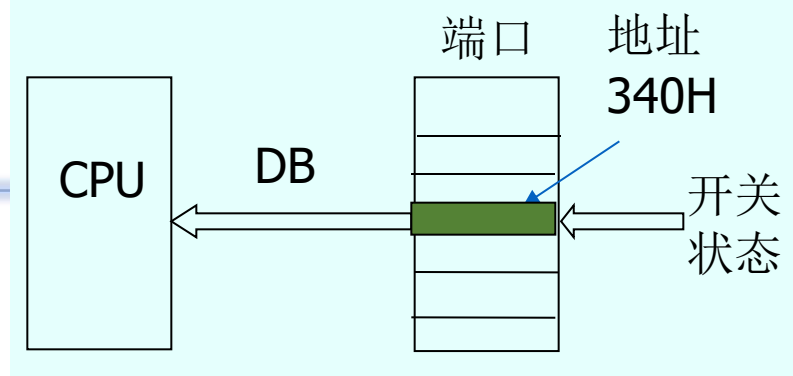
- （1）编程实现LED灯反应开关状态，如：K0闭合则LED0亮，断开则灭。
- （2）编程，测试K3是否闭合，是则所有LED灯亮，否则全灭。

执行IN、OUT指令时，AB、DB和CB信号的变化



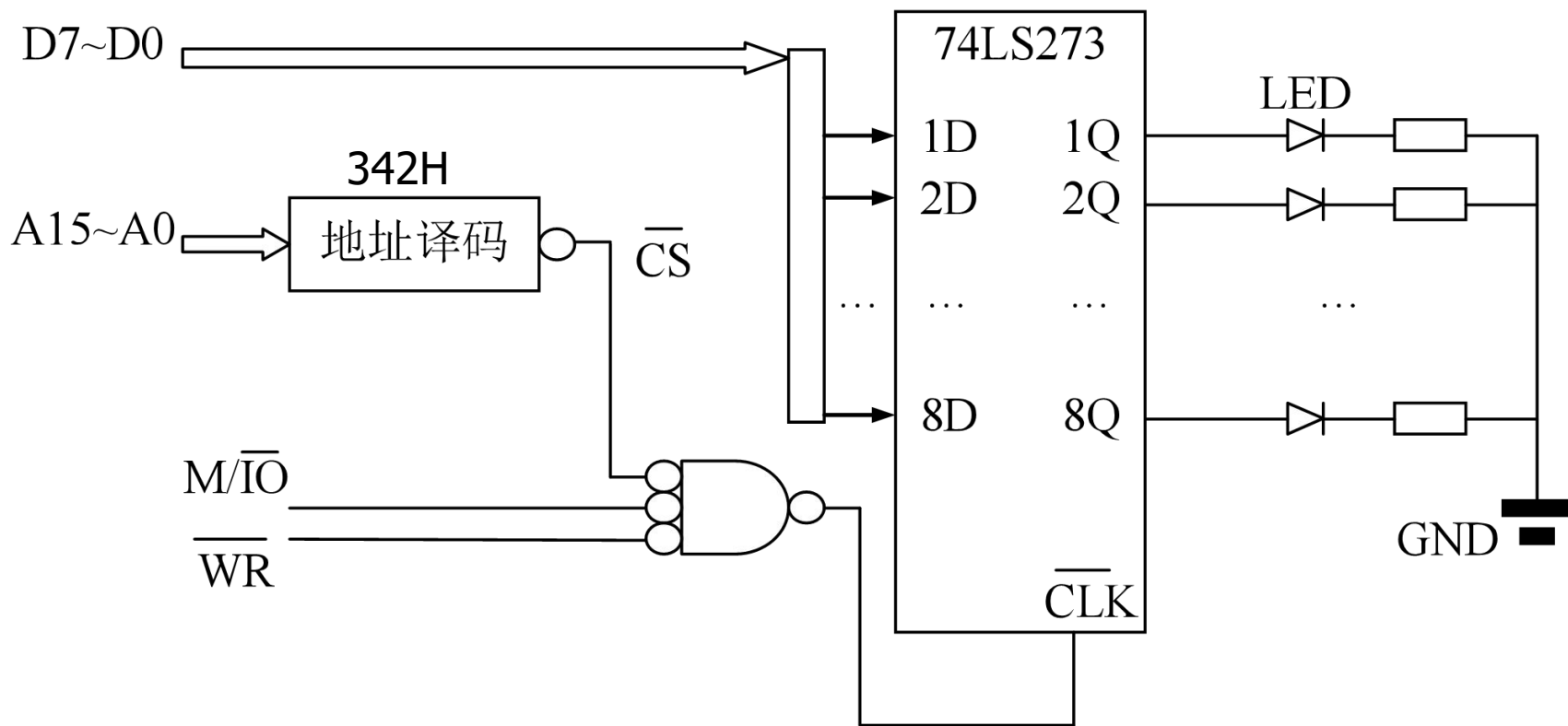
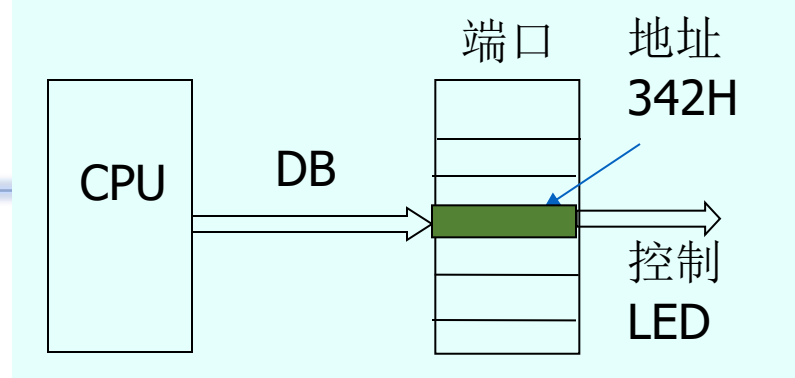


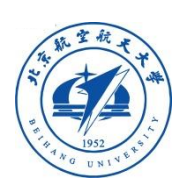
无条件传送 (输入)





无条件传送 (输出)





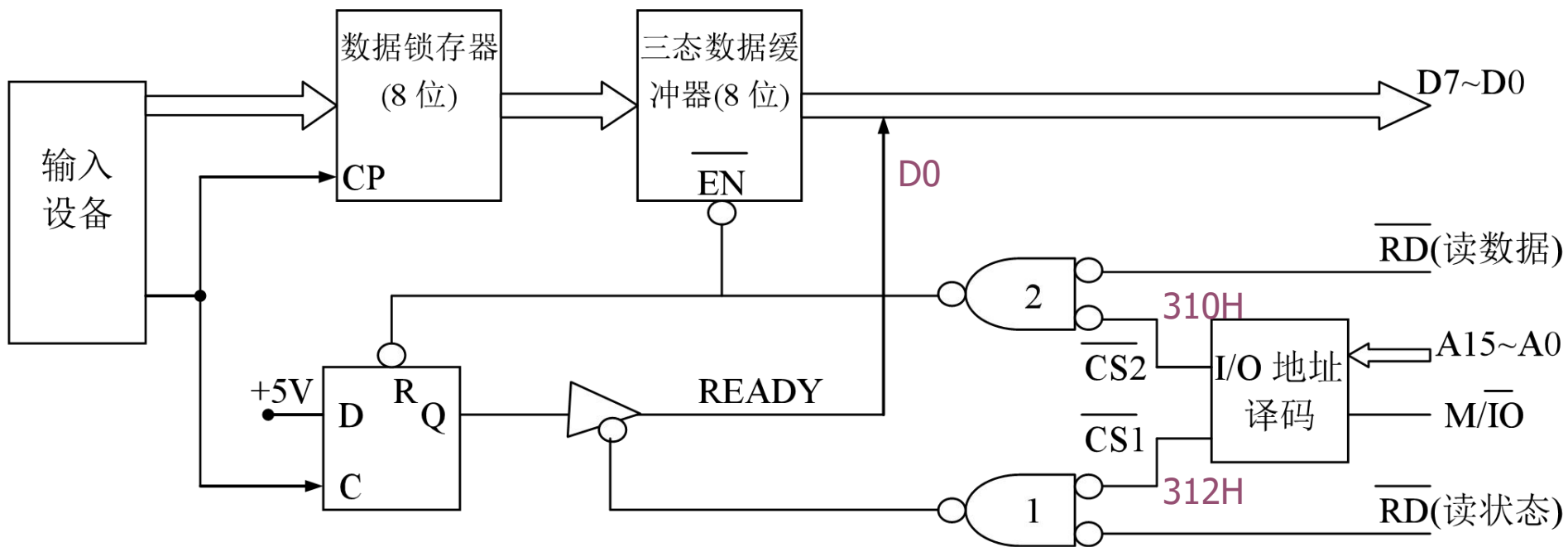
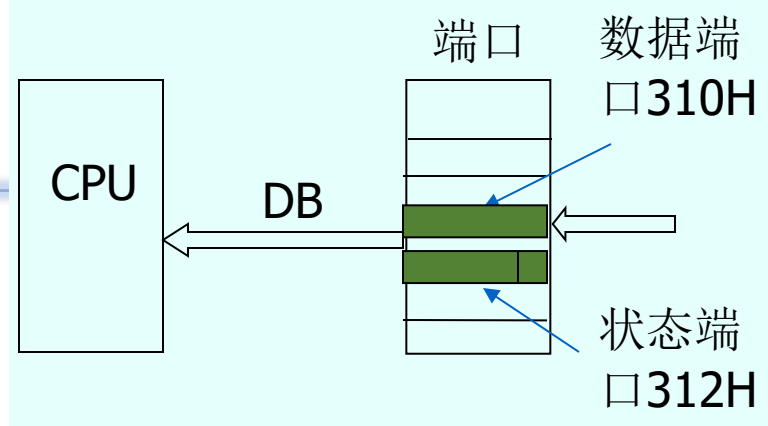
6.1.3 CPU与外设数据的交互-程序方式

2) 条件传送/查询方式

- 数据传送前需要先查询外设的状态。
 - 从外设的状态端口读取它的当前状态（查询外设状态）
 - 若外设没有就绪则继续查询。CPU一直反复查询，不能做别的事情，直至外设准备就绪
 - 当外设准备就绪后，数据才开始传送

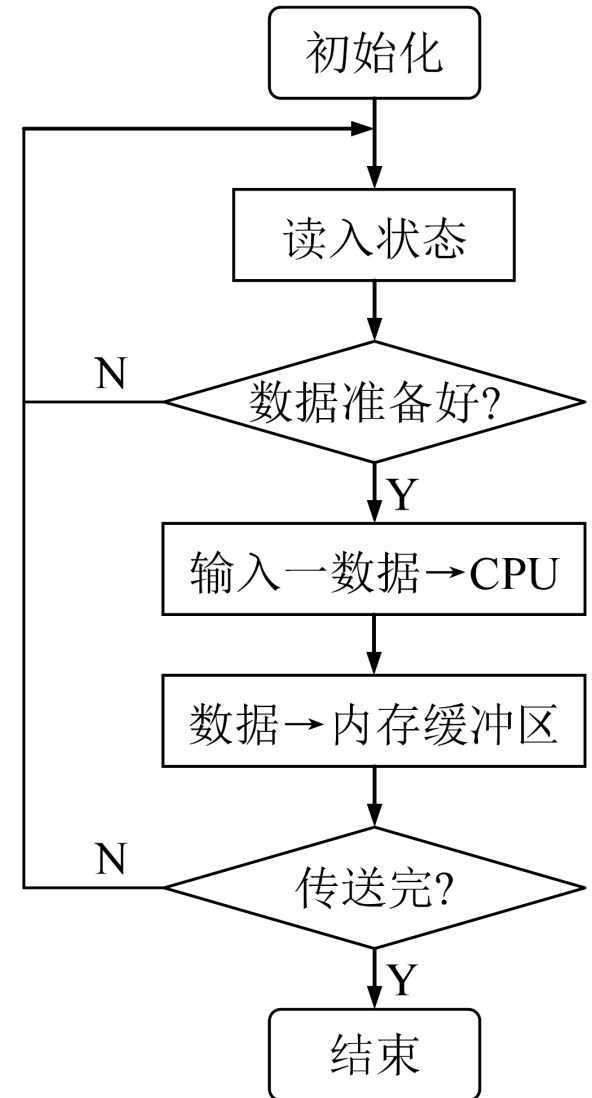


查询方式 (输入)



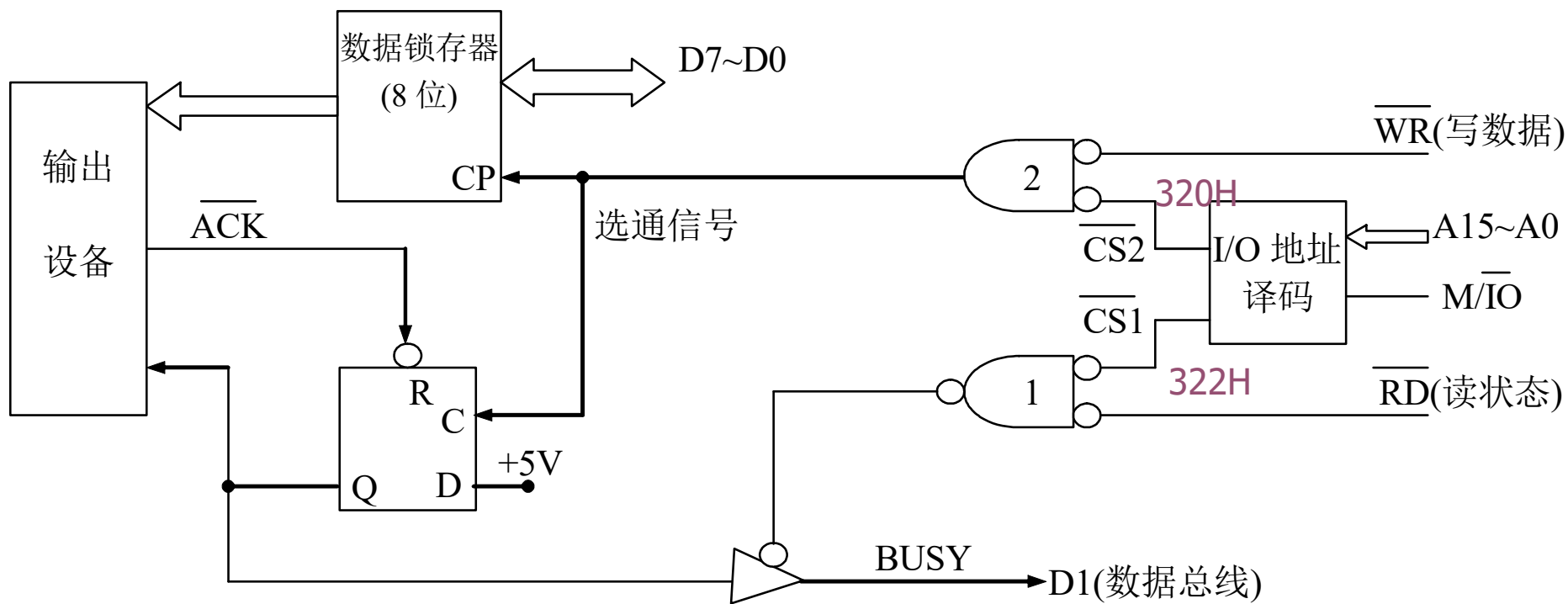
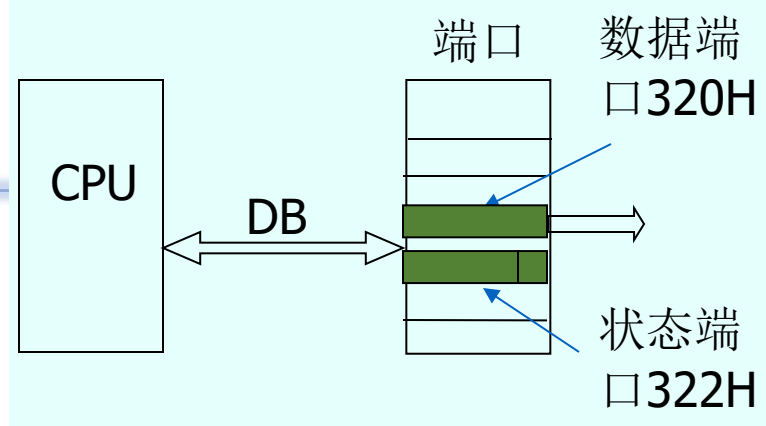
设状态端口的地址为PORT_S1,输入数据端口的地址为PORT_IN, 传送数据的总字节数为COUNT_1, 则查询式输入的程序段为:

```
MOV BX, 0
MOV CX, COUNT_1
READ_S1: IN AL, PORT_S1
TEST AL, 01H
JZ READ_S1
IN AL, PORT_IN
MOV [BX], AL
INC BX
LOOP READ_S1
.....
```



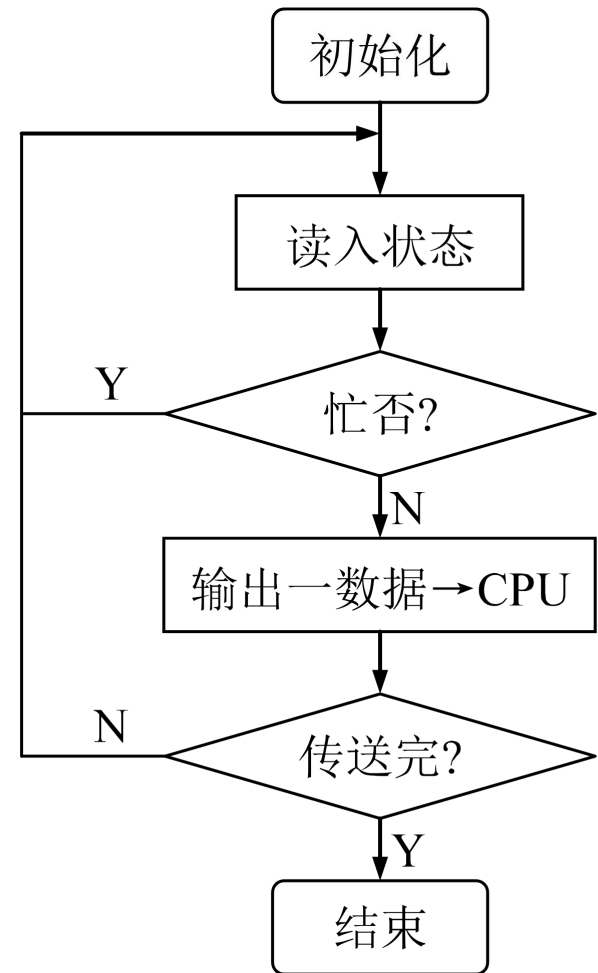


查询方式 (输出)



设状态端口地址为PORT_S2，输出数据口的地址为PORT_OUT，
传送数据的总字节数为COUNT_2，则查询式输出数据的程序段为：

```
MOV CX, COUNT_2  
READ_S2: IN AL, PORT_S2  
TEST AL, 02H  
JNZ READ_S2  
MOV AL, 输出数据  
OUT PORT_OUT, AL  
LOOP READ_S2
```





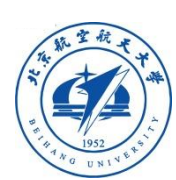
2.中断方式

- 在**查询方式**下真正用于数据传输的时间很少，CPU的**利用率很低**
- 在**中断方式**下，CPU平时可以执行主程序，只有当输入设备将数据准备好之外，或者输出端口的数据缓冲器已经空闲，才向CPU发中断请求。
- CPU响应中断后，暂停当前程序，转去处理管理外设的中断服务程序。等输入或输出结束后，CPU又回去执行原理的程序。
- 中断方式大大提高了CPU的利用率



3. DMA方式

- 中断方式虽然提高了CPU的利用率，但是对于内存和外设之间**大批量数据**传输，仍然显得效率不高
- 为此，可以采用硬件实现在外设与内存之间直接的数据传送，而不通过CPU，这样，数据传送速度的上限就取决于存储器的工作速度。这种方式称为**直接存储器存取**（Direct Memory Access，简称**DMA**）



- DMA方式的数据交换不是由CPU执行一段程序来完成的，而是由硬件实现。数据交换过程由单独的DMA控制器来控制，CPU不介入
- CPU和DMA控制器是共享一套存储器的两个独立的处理单元
- 实现DMA方式而设计的专用控制芯片称为DMA控制器（DMAC），较常用的为Intel 公司的8237A

DMA控制器intel8237A与外设连接的原理示意图

